

Course Outline: Object-Oriented Programming in Python

Title: Object-Oriented Programming in Python **Skill:** Skill 32: Trabajar con datos estructurados de tipo objeto en Python **Learnpack:** + Programación Orientada a Objetos **File:** s32-w11d33-programacion-orientada-a-objetos **Prerequisite:** Skill 31 — Gestionando tablas relacionadas con SQL **Target Audience:** Beginner developers with procedural Python experience **Total Lessons:** 14 **Format:** Mixed (21% hands-on)

Course Description

Students have been writing Python functions and working with SQL — both procedural approaches. This course introduces Object-Oriented Programming: a paradigm where data and behavior are bundled together into objects. Students will learn to define classes, create instances, and apply the four OOP pillars (Abstraction, Encapsulation, Inheritance, Polymorphism) through conceptual lessons and hands-on Python exercises. This prepares them directly for Skill 33, where ORM models are Python classes mapped to database tables.

Course Philosophy

This course emphasizes:

- Mental model shift: from functions-and-variables to objects-and-classes
 - Learning by building: each concept is immediately grounded in Python code
 - Design over syntax: understanding why OOP exists matters more than memorizing keywords
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Classes and Objects	3
02 - Information Hiding	4
03 - Class Hierarchies	4
04 - Assessment	1
05 - Outro	1
Total	14

Learning Objectives:

- Explain the problem that OOP was created to solve
- Describe the mental shift from procedural to object-oriented thinking
- Preview the four OOP pillars and the course structure

Content Outline:

- The limitation of procedural code: as programs grow, functions and variables become hard to organize and maintain
- The OOP insight: group related data and behavior together into objects
- Brief overview of the four pillars: Abstraction, Encapsulation, Inheritance, Polymorphism
- What this course covers and what students will be able to do by the end

Transitions: This lesson motivates the course. Section 01 begins immediately with what a class is and how to write one in Python.

Key Principles:

- Objects bundle data (attributes) and behavior (methods) into a single unit
- OOP models real-world entities as code structures
- The four OOP pillars are tools for writing organized, reusable, and maintainable code

Examples:

- A bank account has data (balance, owner) and behavior (deposit, withdraw) — this is a natural object
 - A **User** in a web app has data (name, email) and behavior (login, update_profile) — another natural object
 - Python's own built-in types are objects: a **list** has data and methods like `.append()`, `.sort()`
-

01.0 Classes and Objects

Learning Objectives:

- Distinguish between a class (blueprint) and an object (instance)
- Recognize how Python uses classes to group data and behavior
- Understand the relationship between class definition and object creation

Content Outline:

- What is a class? A template that describes what data and methods an object type will have
- What is an object? A specific instance created from a class
- The blueprint analogy: a class is the architectural plan; objects are the actual buildings
- How this section unfolds: 01.1 teaches how to write a class in Python; 01.2 is hands-on practice

Transitions: This overview frames the section concept. Lesson 01.1 will go deep into Python syntax: `__init__`, `self`, attributes, and methods.

Key Principles:

- One class can produce many objects, each with independent state
- In Python, everything is an object — integers, strings, lists are all instances of built-in classes
- Thinking in objects: identify the "things" in your domain first, then define their data and behavior

Examples:

- Class `Dog` → objects: `rex = Dog("Rex")`, `buddy = Dog("Buddy")` — two independent instances
 - Class `User` → objects: one per registered user, each with its own name and email
 - Class `Car` → objects: each vehicle in a fleet management system
-

01.1 Defining and Using Classes

Learning Objectives:

- Write a Python class with attributes and methods using the `class` keyword
- Use `__init__` to initialize a new object's state
- Understand the role of `self` in instance methods
- Instantiate objects and call methods on them

Content Outline:

- The `class` keyword: `class ClassName:`
- `__init__`: the constructor — Python calls it automatically when a new object is created
- `self`: the reference to the current instance — always the first parameter of any instance method
- Instance attributes: data stored on the object, set via `self.attribute = value`
- Class attributes: shared across all instances (less common, covered briefly)
- Methods: regular functions defined inside a class, receiving `self` as first parameter
- Instantiation: `my_object = ClassName(arguments)`

Transitions: 01.0 established the mental model. This lesson teaches the exact Python syntax. Next, 01.2 is a hands-on challenge to apply everything learned.

Key Principles:

- `__init__` initializes state; it does not create the object (Python does that before calling `__init__`)
- Every instance method must accept `self` as its first parameter — Python passes the instance automatically
- Instance attributes belong to each object independently; changing one object's attribute does not affect another

Examples:

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
```

```
def bark(self):
    return f"{self.name} says: Woof!"

my_dog = Dog("Rex", "Labrador")
print(my_dog.bark())    # Rex says: Woof!
print(my_dog.name)      # Rex
```

Anti-pattern — forgetting `self`:

```
class Dog:
    def bark():          # Missing self – will raise TypeError when called on an
instance                  instance
        return "Woof!"
```

Every method defined in a class must include `self` as the first parameter. Python automatically passes the instance when you call `my_dog.bark()`, so the method must be ready to receive it.

01.2 Your First Python Class

Challenge Prompt: Define a `BankAccount` class with an owner name and balance, and implement `deposit` and `get_balance` methods that return the updated balance when called.

Solution Code:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

    def get_balance(self):
        return self.balance

account = BankAccount("Alice", 100)
account.deposit(50)
print(account.get_balance())  # 150
```

Challenge Code:

```
class BankAccount:
    def __init__(self, owner, balance=0):
        # store owner and balance as instance attributes
```

```
pass

def deposit(self, amount):
    # add amount to balance and return the new balance
    pass

def get_balance(self):
    # return the current balance
    pass

account = BankAccount("Alice", 100)
account.deposit(50)
print(account.get_balance()) # expected: 150
```

02.0 Information Hiding

Learning Objectives:

- Define information hiding as an OOP design principle
- Explain how abstraction and encapsulation each contribute to information hiding
- Recognize why exposing less leads to safer, more maintainable code

Content Outline:

- Information hiding: classes should expose only what callers need — nothing more
- Two tools that achieve this:
 - Abstraction: hiding HOW something works (complexity)
 - Encapsulation: hiding and protecting the data (state)
- Why it matters: prevents misuse, reduces coupling between components, makes internal changes safe
- Section preview: 02.1 covers abstraction, 02.2 covers encapsulation, 02.3 is hands-on

Transitions: Section 01 taught how to define classes and create objects. Section 02 teaches how to design them well — by controlling what's visible from the outside.

Key Principles:

- Users of a class should not need to know how it works internally
- A clean public interface separates what from how
- Hiding internal state prevents callers from putting objects into invalid states

Examples:

- A car dashboard: you use the steering wheel and pedals (the interface), you don't reach into the engine (the implementation)
- `list.sort()`: you call it and it sorts — the algorithm is hidden and can change without breaking your code
- A web API endpoint: callers send a request and get a response; the database queries behind it are none of their business

02.1 Abstraction

Learning Objectives:

- Define abstraction and explain its role in OOP design
- Design a class that exposes a simple public interface while hiding its internal complexity
- Distinguish between a class's public API and its implementation details

Content Outline:

- Abstraction: reduce complexity by exposing only what a caller needs
- Public interface: the methods and attributes a caller is meant to use
- Internal implementation: how the class actually works — hidden from callers
- Python naming convention: methods starting with `_` are internal by convention
- Abstract base classes (`abc` module): a way to define a required interface without implementing it — mentioned conceptually

Transitions: Abstraction is about what you expose. Next, encapsulation is about how you protect what you don't expose.

Key Principles:

- A class should do one thing well and offer a minimal, clear interface
- The caller only needs the contract: what inputs go in, what outputs come out
- Good abstraction makes classes interchangeable — if two classes share the same interface, a caller can use either without changing code

Examples:

```
class EmailSender:
    def send(self, to, subject, body):
        # Callers use only this method – the rest is internal
        self._connect()
        self._authenticate()
        self._transmit(to, subject, body)
        self._disconnect()

    def _connect(self): ...
    def _authenticate(self): ...
    def _transmit(self, to, subject, body): ...
    def _disconnect(self): ...
```

A caller writes `sender.send("alice@example.com", "Hello", "...")` and never touches `_connect` or `_authenticate`. If the SMTP provider changes, only the internal methods change — the caller's code stays the same.

02.2 Encapsulation

Learning Objectives:

- Explain what encapsulation protects and why it matters
- Use Python's underscore conventions to signal access levels
- Implement a `@property` to control attribute access and add validation

Content Outline:

- Encapsulation: bundling data with the methods that operate on it, and protecting the data from external changes
- The risk of public attributes: callers can set any value, including invalid ones
- Python's convention-based access control:
 - `attribute` → fully public
 - `_attribute` → protected by convention (should not be accessed from outside the class)
 - `__attribute` → name mangling: Python renames it to `__ClassName__attribute`, making accidental access harder
- Properties: `@property` and `@attribute.setter` — provide a clean public interface while keeping the attribute protected

Transitions: Abstraction hid the how; encapsulation protects the data. Next, a hands-on exercise applies both concepts.

Key Principles:

- Python is "consenting adults" — no hard enforcement, but conventions matter and should be followed
- `@property` lets you add validation without breaking the caller's code (they still write `obj.attribute`)
- Encapsulation and abstraction work together: the setter validates; the caller never knows it happened

Examples:

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero")
        self._celsius = value

t = Temperature(25)
t.celsius = 100    # Uses the setter – valid
t.celsius = -300   # Raises ValueError – caught before it corrupts the object
```

Anti-pattern — public attributes with business constraints:

```
class BankAccount:
    def __init__(self):
        self.balance = 0 # Fully public

account = BankAccount()
account.balance = -1000 # Nothing stops this – object is now in an invalid
state
```

Any attribute with a business rule (cannot be negative, must be a valid email, must be within a range) should be protected and accessed via a property with validation.

02.3 Protecting Class Data

Challenge Prompt: Build a `Password` class that stores a hashed version of a password internally and exposes a `verify` method that returns `True` if a given string matches the original password.

Solution Code:

```
import hashlib

class Password:
    def __init__(self, raw_password):
        self.__hashed = self._hash(raw_password)

    def _hash(self, value):
        return hashlib.sha256(value.encode()).hexdigest()

    def verify(self, attempt):
        return self._hash(attempt) == self.__hashed

p = Password("secret123")
print(p.verify("secret123")) # True
print(p.verify("wrongpass")) # False
```

Challenge Code:

```
import hashlib

class Password:
    def __init__(self, raw_password):
        # hash raw_password and store it in a private attribute
        pass
```



```
def _hash(self, value):  
    return hashlib.sha256(value.encode()).hexdigest()  
  
def verify(self, attempt):  
    # return True if hash of attempt matches the stored hash  
    pass  
  
p = Password("secret123")  
print(p.verify("secret123"))    # expected: True  
print(p.verify("wrongpass"))    # expected: False
```

03.0 Class Hierarchies

Learning Objectives:

- Define a class hierarchy and explain what it models
- Explain how inheritance and polymorphism work together
- Understand when a hierarchy adds value and when it adds complexity

Content Outline:

- Class hierarchies: a parent class defines shared behavior; child classes extend or specialize it
- Inheritance: a child class inherits all attributes and methods from its parent
- Polymorphism: objects of different child classes respond to the same method call in their own way
- Why hierarchies matter: eliminate duplication, model real-world specialization, write code that works for entire families of types
- Section preview: 03.1 covers inheritance mechanics, 03.2 covers polymorphism, 03.3 is hands-on

Transitions: Section 02 taught us to protect data inside a single class. Section 03 teaches us to share and extend behavior across families of classes.

Key Principles:

- Hierarchies model specialization: `Animal` → `Dog` → `GoldenRetriever`
- The IS-A rule: only use inheritance when the child IS genuinely a type of parent
- Polymorphism lets you write code that works for any class in the hierarchy, without knowing the specific type

Examples:

- `Shape` → `Circle`, `Rectangle`, `Triangle` — each has its own `area()` method
- `User` → `AdminUser`, `GuestUser` — same login interface, different permissions
- `Vehicle` → `Car`, `Truck`, `Motorcycle` — shared `start_engine()`, different behavior

03.1 Inheritance

Learning Objectives:

- Define a child class that inherits from a parent class in Python
- Use `super()` to call the parent's `__init__` and methods
- Override a parent method to provide specialized behavior in a child class

Content Outline:

- Inheritance syntax: `class Child(Parent):`
- What a child inherits: all methods and attributes defined in the parent
- `super()`: calls the parent class's implementation — essential in `__init__` to set up inherited attributes
- Method overriding: redefining a parent method in a child to change its behavior
- The IS-A rule: use inheritance when the relationship is genuine, not just for code reuse

Transitions: 03.0 set the conceptual stage. This lesson covers the exact Python mechanics. Next, polymorphism shows what happens when you use those overridden methods from the outside.

Key Principles:

- `super().__init__()` must be called in the child's `__init__` to ensure the parent initializes its attributes first
- A child class that overrides a method replaces the parent's version — the child version runs instead
- Inheritance depth: prefer shallow hierarchies (1-2 levels); deep chains become hard to follow

Examples:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self):
        return f"{self.name} says: Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says: Meow!"

dog = Dog("Rex")
cat = Cat("Whiskers")
print(dog.speak())    # Rex says: Woof!
print(cat.speak())    # Whiskers says: Meow!
```

Anti-pattern — deep inheritance chains:

A → B → C → D → E

Each level adds complexity and makes behavior harder to trace. If you're going more than 2-3 levels deep, consider flattening the hierarchy or switching to composition.

03.2 Polymorphism

Learning Objectives:

- Define polymorphism and explain why it makes code flexible and extensible
- Write a loop that calls the same method on objects of different types
- Explain duck typing and how Python implements polymorphism without enforcing types

Content Outline:

- Polymorphism: "many forms" — the same method call produces different results depending on the object's type
- How it works: child classes override parent methods; callers call the method without knowing the specific type
- A polymorphic loop: iterate over mixed-type collections, call the same method on each
- Duck typing in Python: if an object has the method, it works — no explicit type hierarchy required
- `isinstance()`: when you genuinely need to check the type (keep this rare)

Transitions: Inheritance gives child classes their own version of a method. Polymorphism is what that looks like from the caller's perspective — one call, many behaviors. Next: a hands-on exercise that builds a class hierarchy and exercises it polymorphically.

Key Principles:

- Polymorphism eliminates type-checking if/else chains: instead of `if type == "dog": ...`, just call `animal.speak()`
- Duck typing: "if it walks like a duck and quacks like a duck, it's a duck" — Python checks for the method, not the type
- New types can be added to a hierarchy without changing any calling code

Examples:

```
animals = [Dog("Rex"), Cat("Whiskers"), Dog("Buddy")]
for animal in animals:
    print(animal.speak())
# Rex says: Woof!
# Whiskers says: Meow!
# Buddy says: Woof!
```

Anti-pattern — manual type checking:

```
for animal in animals:
    if type(animal) == Dog:
        print(animal.name + " says: Woof!")
    elif type(animal) == Cat:
        print(animal.name + " says: Meow!")
```

This defeats polymorphism. Every time a new animal type is added, this code must be edited. The polymorphic version never needs to change.

03.3 Extending Classes

Challenge Prompt: Create a `Shape` base class with an `area` method, then implement `Circle` and `Rectangle` subclasses that override it and return the correct area when called.

Solution Code:

```
import math

class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(round(shape.area(), 2))
# 78.54
# 24
```

Challenge Code:

```

import math

class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, radius):
        # store radius
        pass

    def area(self):
        # return pi * radius squared
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        # store width and height
        pass

    def area(self):
        # return width * height
        pass

shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(round(shape.area(), 2))
# expected: 78.54, then 24

```

04.0 OOP Knowledge Check

Learning Objectives:

- Assess understanding of classes, objects, and the four OOP pillars
- Evaluate ability to distinguish between abstraction, encapsulation, inheritance, and polymorphism
- Confirm understanding of Python-specific OOP mechanics (**self**, **super**, underscore conventions, properties)

Question Format: Multiple choice

Questions:

1. Which of the following best describes a class in Python? a) A function that returns an object b) A blueprint that defines the structure and behavior of objects ✓ c) A variable that holds multiple values d) A module that contains utility functions
2. What does `__init__` do in a Python class? a) Destroys the object when it is no longer needed b) Defines the class name c) Initializes the object's attributes when an instance is created ✓ d) Imports external modules

3. What is the purpose of `self` in a Python class method? a) It refers to the class itself b) It refers to the current instance of the class ✓ c) It is required by Python syntax but has no effect d) It holds all class attributes globally
4. Which OOP pillar describes hiding the internal implementation of a class and exposing only what the caller needs to use it? a) Encapsulation b) Polymorphism c) Inheritance d) Abstraction ✓
5. A `Temperature` class stores `_celsius` as a protected attribute and validates values through a `@property` setter. Which pillar does this primarily demonstrate? a) Abstraction b) Encapsulation ✓ c) Inheritance d) Polymorphism
6. What does `super().__init__()` accomplish inside a child class's `__init__` method? a) Creates a new instance of the parent class b) Deletes the parent class c) Calls the parent class's `__init__` to initialize inherited attributes ✓ d) Overrides the parent class completely
7. A list contains `Dog`, `Cat`, and `Bird` objects. A loop calls `animal.speak()` on each, and all work correctly without any `if/else` type checks. This is an example of: a) Encapsulation b) Abstraction c) Polymorphism ✓ d) Inheritance
8. Which is the correct Python syntax for defining a child class? a) `class Child inherits Parent:` b) `class Child extends Parent:` c) `class Child(Parent):` ✓ d) `class Child -> Parent:`
9. In Python, what does a single underscore prefix (`_attribute`) signal? a) The attribute is private and Python blocks all external access b) The attribute is protected by convention and should not be accessed from outside the class ✓ c) The attribute is deleted after the object is created d) The attribute belongs to the class, not the instance
10. What is duck typing? a) A way to verify an object's type before calling a method on it b) A Python approach where objects are used based on their methods and attributes, not their explicit type ✓ c) A pattern for creating classes that inherit from multiple parents d) A method for encrypting class attributes

Evaluation Criteria: Questions cover core OOP terminology, Python class syntax, the four pillars and their distinctions, and Python-specific mechanics (`self`, `super()`, naming conventions, properties).

Score Interpretation:

- 9–10 correct: Strong OOP foundation — ready to apply these concepts with an ORM in the next skill
- 7–8 correct: Good understanding with minor gaps — review the sections where questions were missed
- 5–6 correct: Foundational gaps — revisit the relevant sections before continuing
- Below 5: Recommend repeating the course before proceeding to Skill 33

05.0 Conclusion

Content Outline:

- Course recap — what students accomplished:
 - Defined Python classes with attributes, constructors, and methods
 - Understood the class/object relationship: blueprint versus instance
 - Applied all four OOP pillars: Abstraction, Encapsulation, Inheritance, Polymorphism
 - Wrote working Python code using class hierarchies and polymorphic method calls
- Connection to bigger picture: OOP is the foundation of modern backend development. In the next skill, you will use SQLAlchemy — an ORM where every database table is represented as a Python class. Every **Model** you write IS an OOP class, and the ORM relies entirely on the concepts you just learned.
- Next steps: Skill 33 introduces Object-Relational Mapping, where Python classes become database models and OOP principles drive how you interact with data
- Resources for further exploration:
 - Python official docs — Classes: <https://docs.python.org/3/tutorial/classes.html>
 - Real Python — OOP in Python 3: <https://realpython.com/python3-object-oriented-programming/>
- Encouragement: You have made one of the most important conceptual leaps in software development — from writing functions to designing objects. This mental model will stay with you throughout your career as a developer.

Note: This is conclusion only — no theory, exercises, or assessment.
